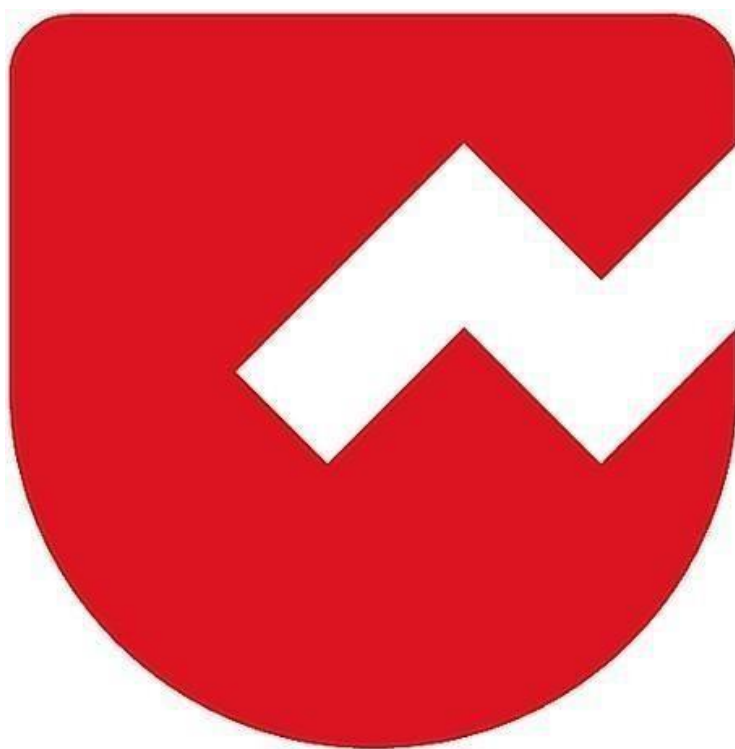


Zoho Schools for Graduate Studies



Notes

Java Program to read line by line and write line by line

```
import java.io.*;

public class ReadLineDemo {

    public static void main(String[] args) throws Exception {

        String srcPath = "input.txt";

        FileReader fr = new FileReader(srcPath);

        BufferedReader br = new BufferedReader(fr);

        String destPath = "output.txt";

        FileOutputStream fos = new FileOutputStream(destPath);

        PrintWriter pw = new PrintWriter(fos);

        String value = br.readLine();

        while (value != null) {

            pw.println(value);

            value = br.readLine();

        }

        fr.close();

        br.close();

        fos.close();

        pw.close();

        System.out.println("File content are copied line by line.");

        Runtime rt = Runtime.getRuntime();

        String command = "open " + srcPath;

        Process p = rt.exec(command);

    }

}
```

FileReader:

- FileReader is a high-level character input stream used to read characters from a file. It reads data character by character.
- Input: A file path (String) or a File object.
- FileReader is used here to open the source file for reading.

BufferedReader:

- BufferedReader wraps around a Reader (like FileReader) and provides buffering, which makes reading efficient. It also provides the readLine() method to read one line at a time.
- Input: A Reader object (FileReader).
- readLine() → reads a line of text
- read() → reads a single character

PrintWriter:

- PrintWriter is a high-level character output stream used to write text to files. It provides convenient methods like print() and println().
- Input: OutputStream like FileOutputStream.
- print(String s) → writes text without newline.
- println(String s) → writes text with newline.
- close() → flushes data and closes the stream.

Runtime:

Runtime class in the java.lang package. The Runtime class represents the Java application's runtime environment. Every Java program has one instance of Runtime, which can be obtained using the method

```
Runtime rt = Runtime.getRuntime();
```

The Runtime object allows the program to perform system-level operations, such as executing external commands. Runtime acts as a bridge between your Java program and the operating system. You cannot create multiple Runtime objects; you always use the single instance provided by `getRuntime()`.

exec():

- The `exec()` method is a **member of the Runtime class**. It is used to **run system commands** from within a Java program.
- You pass a **command string** to `exec()`, for example "open input.txt" (on Mac) or "notepad input.txt" (on Windows).
- The command is executed by the operating system.
- The method returns a **Process object** representing the program or command that is running.

```
Process p = rt.exec("open input.txt");
```

Java Program to play a video or audio file

```
import java.io.*;

public class PlayVideoFileDemo {

    public static void main(String[] args) throws Exception {

        String srcPath = "path";

        Runtime rt = Runtime.getRuntime();

        String command = "open " + srcPath; // for macOS

        Process p = rt.exec(command);

    }

}
```

Java Program to get input from multiple streams

```
import java.io.*;

public class SequenceInputDemo {

    public static void main(String[] args) throws Exception {

        byte[] b = "It was all yellow".getBytes();

        ByteArrayInputStream bis = new ByteArrayInputStream(b);

        SequenceInputStream sis = new SequenceInputStream(System.in, bis);

        String destPath = "output.txt";

        FileOutputStream fos = new FileOutputStream(destPath);

        int value = sis.read();

        while (value != -1) {

            fos.write(value);

        }

    }

}
```

```
        value = sis.read();
    }

    bis.close();
    sis.close();
    fos.close();
    System.out.println("Keyboard and byte array content are copied.");

    Runtime rt = Runtime.getRuntime();
    String command = "open " + destPath;
    Process p = rt.exec(command);
}
}
```

SequenceInputStream:

SequenceInputStream is a special input stream that joins two or more InputStreams together and makes them behave like one continuous stream.

It reads all data from the first stream, and when it finishes, it automatically continues reading from the second stream.

Serialization:

Serialization in Java is the process of converting an object into a byte stream so it can be stored in a file or transferred across a network. To enable this, the class must implement the `Serializable` interface. Java provides `ObjectOutputStream` to write the object and `ObjectInputStream` to read it back during deserialization. This allows you to save and restore the complete state of an object.